

SPECTRAVIDEO

USERS GROUP WELLINGTON

Wellington SV Users Group

Committee Members

Chairman ----- Darryl Allison Ph. 45-522 (Paraparaumu)

Treasurer ----- Ross Fletcher 788-454

Don Stanley 896-373

Scotty Baxter 784-615

Jim Oliver 895-618

Committee Members

Steve Hanson 896-157
May / June 1986

Joseph Del 896-373

Other specific areas of responsibility for committee members are

Don -- Software
Jim -- Media (audio, video, tape software)
Scotty -- Library

Darryl

Games Scores	2
MSX to SV Conversions	4
Exploring Basic Part 6	7
Questions & Answers	13
Program - 3 Dimensional Tic Tac Toe	15
MSX Software	21
Classifieds	25

Just a reminder to check your subscription expiry date which is printed at the bottom of your newsletter label.

It is your subscriptions which keep the club running, therefore if you want the support of the group to continue, please help us by checking your subscription renewal date.

S P E C T R A V I D E O

U S E R S G R O U P -- Wgtn

Volume 3, Number 6 -- May/June 1987

The next meeting is on Monday, June 22, 1987 at 7.30pm in the Staff room of Newlands Intermediate, Bracken Rd, Newlands. The July meeting will be the 20th of July, 1987. (ie as usual the 3rd Monday of the month.)

Wellington SV Users Group

Committee Members

Chairman ----- Darryl Alison Ph 85-522 (Paraparaumu)
Treasurer ----- Ross Fletcher 788-454
Don Stanley 896-379
Scotty Baxter 784-619
Jim Oliver 699-618

Newsletter Committee

Steve Hanson 843-495
Joseph Dol 889-275

Other specific areas of responsibility for committee members are
Don -- Software Consultant
Jim -- Radio Access & public domain tape software;
Scotty -- Library;
Darryl -- Program evaluation committee chairman & Kapiti users representative.
Ross -- Public Domain CP/M

Club Subscriptions

Just a reminder to check your subscription expiry date which is printed at the bottom of your newsletter label.

It is your subscriptions which keep the club running, therefore if you want the support of the Group to continue, please help us by checking your subscription renewal date.

If your subs are due before July, please forward them to the Club's P.O. Box 26050 - Newlands, WELLINGTON.

Remember - the subs are only:

\$18.00 For NZ Residents
NZ\$30 For Australasian Subscriptions
NZ\$40 For other areas overseas.

GAMES SCORES

Spectron	12910	Robert Campin
Sector Alpha	2445200	Garry Clark
Kung Fu	82435	Garry Clark
	48705	Garry Clark (*)
S C Force	1640000	Alexander Lindsay
Telebunny	113520	Garry Clark
	32720	Garry Clark (+)
	31670	Kalima Collinson (*)
Omac	35360	Richard Bremford
	25700	Don Stanley (+)
Ninja	251350	Nicola Tuinman
Tetra	579650	Scottie Baxter
Turboat	68390	Garth Thornton (*)
Turboat	104610	Taane Clark
Sasa	172950	Steve Lacey
Frantic Fred	21720	Rachel Hughes (level4)
Flipper Slip	37640	Garth Thornton
Bats Alive	3540	Colin Chin (*)
Galaxy	1724	Scottie Baxter
Jumping Jack	2600	Colin Chin (*)
TV Nightmare	7040	"
	3640	" (*)
	4980	Deirdre Chin (+)

(+) = started in normal level
(*) - started in arcade level
otherwise practice level or unknown.

Public Domain CP/M

Anyone wanting anything from the library of CP/M PD software, please write to Ross Fletcher at the club address, specifying either which disc, or which programs you would like. The listings in previous issues are printed from double sided disks

therefore as most drives are single sided, a whole disc from the list now occupies up to 2 full discs. Most of the programs on the discs are 'Squeezed', this means they have been run through a function in New Sweep (NSW, Nsweep etc. but not the earlier Sweep) so as to take up as little room as possible. They all have a 'Q' as the 2nd letter in their extension, and can be easily unsqueezed by using New Sweep to tag them and then the 'Q' option to unsqueeze them, usually onto another disc. Please send discs, rather than asking us to purchase them for you, and please send them formatted. Please also include, either a stamped return mailer, or add 60c to your payment to cover postage. We will return your disk in your packaging.

Newsletter Contributions

Again our thanks go to the members who are regular contributors for their articles. We would however, like to hear from those people out there who have never written anything for the newsletter. So how about it.

There are a lot of you who are writing programs in BASIC that other people would be interested in, so send in a brief outline of your program together with your program (on cassette or disk and we can merge it into the newsletter) care of the club PO Box. Also any of the following items would be gladly received:

- Programs - Basic, Pascal, Assembly, You Name It.
- Programming hints, and techniques.
- Utilities - any type, as long as they are self explanatory or come with instructions.
- News - anything you think other club members might like to know.
- Problems - we will find someone to answer them if we can't.

Everything we receive will be considered. If you are sending in printed copy (we prefer a disk or tape file) make sure that

1. It is printed on A4 paper with 6 lines at the top and bottom and 1 inch on either side.
2. you use a ribbon that gives dark print or use emphasised mode

otherwise it will not be possible to print your article.

By Jim Oliver

Further investigation showed that the I/O error was occurring immediately after the data stream ended. The programs could be LISTED or even LLISTed but NOT RUN or CSAVED. I don't like typing that much so the print outs were of academic interest only at this stage.

```
FOR T=&H8001 TO &H8001+50:LPRINT HEX$(PEEK(T));" ";:NEXT T
```

After the computer has booted you must enter in immediate mode without line numbers:

and so on until you get to &h800F.

The storage of a Basic program in the computer is a bit tricky to explain quickly and has been covered already by Don Stanley (many thanks Don) in SVUG Wgtn. newsletter No.7 page 5. In theory when the computer is warm booted only the first 15 bytes from &H8001 are changed so that most of the original program is still there quite intact. In theory. In practise a few more bytes COULD be upset, particularly after an error occurs.

Wellington SV Group Newsletter

May/June 87

PAGE 4

(from &h8003) 08,54,0,C5,32,76,90,0,0,0,0 up to &h800D.

Anyone who has done this sort of thing will know these bytes well, the computer ROM produces them with monotonous regularity. Note that bytes 8004 and 8005 when reversed give the hex value for 21512, (&H8054) the next line no. In this case a line number produced by that tricky ROM and one which fools the various basic instructions into not working.

Once you have repoked the correct bytes you can then (with luck) save the file in ASCII format by using SAVE "1:filename",A. Don't try the normal tokenised save, it won't work if 21512 is the first line of the file. The error line number will be saved in the ASCII file but must be removed, read on.

You should now have an ASCII file with all the goodies in it.

If you look at this file (use FILE, SVUG Wgtn. Newsletter Sept. 1986 p. 6-10) you will see that some funny things have happened. A lot of the key words have mysteriously changed. At this point your LLIST print out of the original program begins to take on a new importance.

If you have Wordstar or a similar CP/M word processor the next task is fairly easy. Even if you only have Just Write Jnr. you can get by.

If you have WS then firstly convert the Basic ASCII file to a CP/M one using CPM.IT (SVUG Wgtn. No.6. page 13). With Just Write you don't need to convert to CP/M at all and you can edit the file as normal.

Once in the edit mode you need to use the 'find and replace mode' to automatically alter the file to match the original print out. With Wordstar always choose the 'N' option, never 'D' to start the edit session when handling files as distinct from documents.

All this is necessary since the MSX ROM records key words using one set of hex tokens and SV uses a different set, so equals (=) comes out as the key word TIME when the MSX program in the computer at the time of the warm boot goes under the control of SV ROMs. There are others as well and that's ~~what~~ we will talk about now. Lets now examine the table at the end of the article.

The keyword entered under MSX mode (in the MSX column) becomes, after a warm boot, the keyword under the SV mode (in the SV column). All the other tokens I have looked at do not change in SV mode.

Of course there are few others which you need to watch for e.g different SCREEN modes, use LOCATE in place of PSET or PRESET,

omit the OPEN "GRP" and CLOSE statements, and the PRINT #1, is replaced with PRINT etc. For this sort of thing your original printout is essential.

When you do your 'find and replace' follow this next scheme and you should at least get the arithmetical tokens correct. In many ways this is the most difficult part of converting MSX to SV.

First	Find	/	& Replace with	AND	use GN option
Second	"	-	" " "	/	" GN "
No. 3	"	<	" " "	-	" " "
No. 4	"	>	" " "	<	" " "
No. 5	"	SPRITE	" " "	>	
No. 6	"	+	" " "	*	
No. 7	"	=	" " "	+	
No. 8	"	TIME	" " "	=	
No. 9	"	^	" " "	OR	
No. 10	"	MON	" " "	TIME	
No. 11	"	PRINT#1,	" " "	PRINT	
No. 12	"	KEY	" " "	SPRITE	
No. 13	"	MOTOR	" " "	KEY	
No. 14	"	EQV	" " "	MOD	

Remember that the REM statements are not tokenised and as such are unchanged when you boot SV, so if there are a few ANDs used in these statements (quite common) these will end up as XOR if you choose the no prompt option under WS (GN). Also the KEY part of INKEY\$ will be changed to INSPRITE\$ if you don't choose the wholeword option (W). If in doubt always choose the GW option and follow the program on the printout as your word processor works though the file.

Whew! Its a lot of work, but you will learn to love Wordstar by the end of it all.

I have converted quite a few of the original MSX P.D files now and I will be putting them onto the SV P.D tapes and discs. I have converted them to the stage that they will at least run, however some of the French is beyond me, so I'll leave that interpretation up to you.

MSX	BECOMES	SV	MSX	BECOMES	SV
BASE	>>>>>>	SWITCH	BSAVE	>>>>>>>>	DIAL
BLOAD		MDM	CALL		MAX
EQV		OR	IMP		XOR
KEY		MOTOR	MOTOR		BSAVE
MOD		EQV	MAX		BLOAD
OR		^	SPRITE		KEY
TIME		MON	VDP		CLICK
XOR		AND	AND		/

/	- (minus)		-	<
<	>		>	SPRITE
*	+		+	=
=	TIME			

p.s If you are interested in the key word comparisons use this:

```

5 SCREEN ,0 : CLS
10 X=&H2C9:SL$="A"      (&H3A80 under MSX)
20 NM$ = SL$
30 P = PEEK(X)
40 IF P > 128 THEN 90
50 IF P = 0 THEN 130
60 NM$ = NM$ + CHR$(P)
70 X = X + 1
80 GOTO 30
90 P = P - 128
100 NM$ = NM$ + CHR$(P)
110 PRINT NM$;" ";
120 X = X + 2: GOTO 20
130 SL$ = CHR$(ASC(SL$)+1):X = X + 1
140 IF SL$ > "Z" THEN END ELSE 20

```

This courtesy of Mr.D. Whitmore (Peeks and Pokes for the SPECTRAVIDEO Pub TNL 1984)

Jim suggests that you take a look at the latest Bits & Bytes and also Computer Forum Voll. #11 for the articles on MSX--->SV

Exploring Basic Pt-6

By L.A. Dunning

This month, I describe SCREEN 2 and how SPRITE's are used.

MULTICOLOUR (SCREEN 2)

SCREEN 2 works in an entirely different manner than that of SCREEN 1. When I first saw this mode I assumed that it ~~was~~ a software controlled version of SCREEN 1, but nothing could be further from the truth.

The display is in multicolour mode when bit 1 of register 0 and bit 4 of register 1 are reset and bit 3 of register 1 is set. In this mode the screen is divided into a 64 x 48 grid of colours, each the equivalent of a 4 x 4 pixel block in graphics II. In this mode, only the Pattern Name Table

(0800H) and Pattern Generator Tables (0000H) are used, the Colour Table is not required. The relationship between the two tables is a bit more complex than the previous mode. Each entry in the NTB describes a pattern block for that position. Each position defines a 2 x 2 colour block producing a 8 x 8 SCREEN 1 pixel area on the screen. Diagram 1 illustrates this relationship. There are 32 columns by 24 rows. This produces a total of 768 positions, the same number as in graphics II.

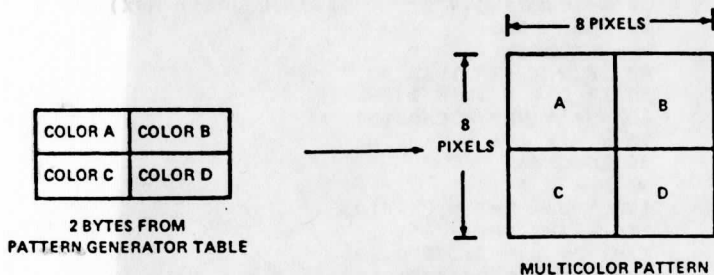


Diagram 1

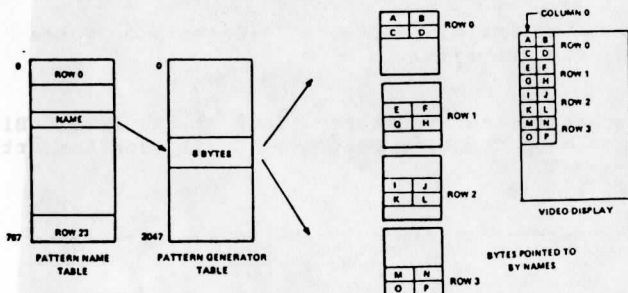


Diagram 2

As might be expected, a byte in the NTB indicated which block of 8 bytes on the PGB is used, however it is not a straight one-one relationship as in graphics II. Only 2 bytes of each 8 byte pattern is used for each position. Refer to diagram 2.

Since there are only 16 different colours, only 4 bits or half a byte is needed for each pixel. Thus the first byte is used for pixels A & B, the second byte for pixel C & D. The 4 most significant bits indicate A & C, the least indicate B & D. Which 2 bytes of the pattern block are used? This is dependent upon the row the character position is in. If the row is 0, 4, 8, 12, 16 or 20 the first 2 bytes are used, if the row is 1, 5, 9, 13, 17 or 21 the second 2 bytes are used,

it the row is 2, 6, 10, 14, 18 or 22 the third 2 bytes are used, if the row is 3, 7, 11, 15, 19 or 23 the last 2 bytes are used. Diagram 3 illustrates this relationship.

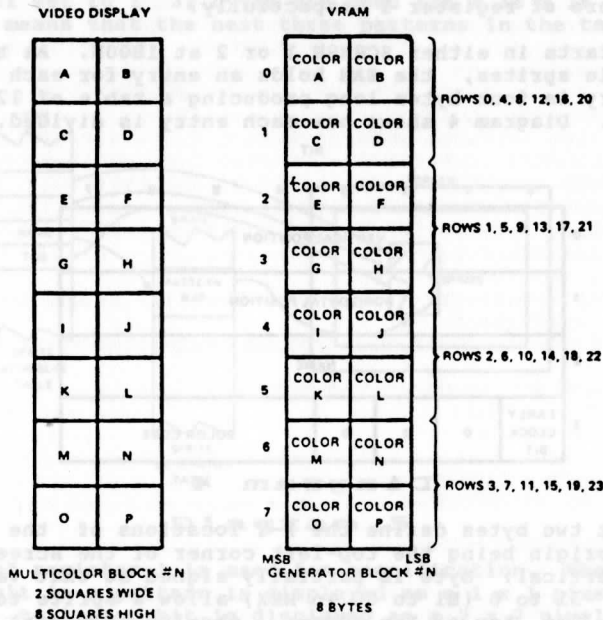


Diagram 3

This sounds more complicated than it is. The net effect is that 4 character positions, or 16 pixels, can be defined using 8 bytes of the PGB. To simplify matters listing 1 gives you a visual example of the various relationships.

It would first appear that SCREEN 2 is in all ways inferior to SCREEN 1, since only a lower definition can be obtained. This is balanced by two advantages; firstly SCREEN 2 only takes up 4K of VRAM, enabling up to four pages being stored at any time. By juggling the registers you could produce a short animation; secondly GET & PUT work properly on this mode. Listing 2 demonstrates a slow scroll on SCREEN 2 using the SPRITE bug listed in an earlier article.

SPRITES

Sprites can be used in all modes except text mode. In this mode they are blanked out by the VDP. The appearance and location of sprites is dependent upon two tables; the Sprite Attribute Table (SAB) and the Sprite Pattern Generator Table (SPGB). The size and magnification is also dependent upon bits one and zero of register 1 respectfully.

The SAB starts in either SCREEN 1 or 2 at 1B00H. As there are 32 possible sprites, the SAB holds an entry for each sprite; each entry is four bytes long producing a table of 128 bytes in length. Diagram 4 shows how each entry is divided.

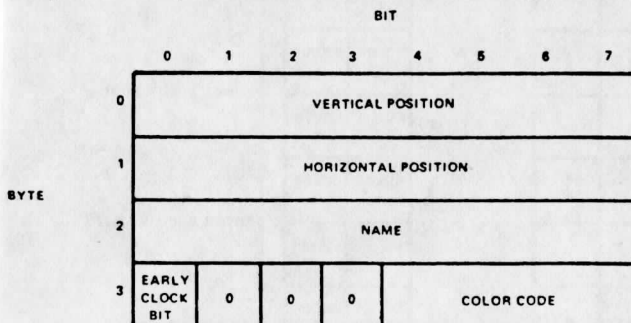


Diagram 4

The first two bytes define the X-Y locations of the sprite, with the origin being the top-left corner of the screen. The first (vertical) byte is partially signed so that values of between, -31 to 0 (E1 to 00 in HEX) allow a sprite to "bleed in" from the top of the screen, showing only part of the sprite. Values about 191 will do this at the bottom of the screen. A value of 208 (D0 HEX) in this byte will blank out the sprite but it will also truncate sprite processing so that sprites after that one will not be processed.

The second (horizontal) byte is not signed like the first and defines 256 positions horizontally. How then does the sprite "bleed in" from the right? The most significant bit of the fourth byte is called the Early Clock (EC) bit. When this is set to 0, the horizontal position of the sprite is shifted to the left by 32 pixels.

The third byte points to the pattern used in the SPGB. This table is explained below. The fourth byte uses the least significant bits to define the colour of the sprite and the most significant bit is the early clock bit as mentioned above. The remaining bits are unused.

The SPGB is normally set to 3800H by basic. There are 256 pattern definitions in this table, each is 8 bytes long producing a table of 2048 bytes. The third byte in the SAB is

used to point to a pattern position in this table and can be any of these patterns. Diagram 5 illustrated the relationships between the two tables.

Bit 1 in register 1 (size) affects how the patterns are displayed. If set to 0, only 8 bytes are used for an 8 x 8 pattern; if set to 1 32 bytes are used for a 16 x 16 pattern, and this means that the next three patterns in the table are also used.

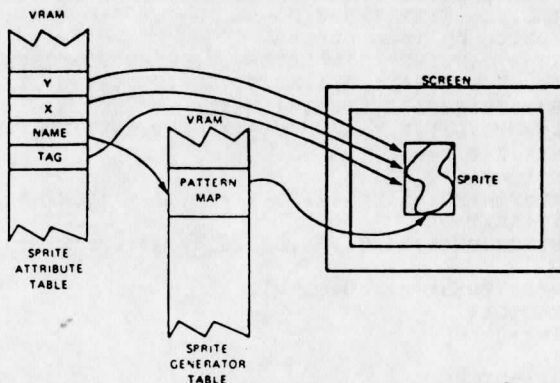


Diagram 5

Bit zero of register 1 is used for magnification. When set to 0, each bit in a pattern is displayed as a 1 x 1 pixel block; when set to 1 each bit is displayed as a 2 x 2 pixel block. The important thing to remember is that size and magnification apply to ALL sprites displayed. You cannot mix sizes or magnifications at the same time. The way round the limitation is to set the size to 16 x 16 and then enlarge a pattern on a bit basis. Listing 3 illustrates manipulation of the sprite tables to produce desired effects.

LISTING 1

By L A Dunning

```

1 REM Listing 1, Part 6
2 REM Demonstrates Relationships in SCREEN2
3 CLEAR2000:DEFINT A-Z:SCREEN2:NT=&H800:PG=0
4 REM Set up colours in first two pattern definitions
5 FORA=0TO15:VPOKEPG+A,A*A*16:NEXT
6 REM Poke in horizontal line by using pattern #
7 FORA=0TO31:VPOKEKNT+A,0:NEXT
8 REM Poke in vertical line by using pattern #
9 FORA=0TO24:VPOKEKNT+4+A*32,1:NEXT
10 IFNOT(STRIG(0))GOTO10
11 SCREEN1:LIST:REM You can list in SCREEN1, not very useful

```

but interesting to know. Try changing line 11 to SCREEN2 instead of SCREEN1. Press stop key when you see this line.

12 REM

LISTING 2

By L A Dunning

```

1 REM Listing 2, Part 6
2 REM Demonstrates scrolling in SCREEN2
3 REM Initialisation
4 CLEAR2000:J=RND(-TIME):DEFSTRA:DEFINTV:SCREEN2,2:XL=40:YL=3
  0:DIM A(18),V(XL,YL):NT=&H800:PG=0
5 REM Putting patterns into strings
6 PRINT1":LOCATE32,0:PRINT"2":LOCATE64,0:PRINT"3":FORB=16TO1
  8:A(B)=RIGHT$(SPRITE$(48+B),31)+LEFT$(SPRITE$(49+B),1):NEXT
  8 FORB=0TO15:A(B)=STRING$(32,B+B*16):NEXT
10 FORX=0TOXL:FORY=0TOYL:V(X,Y)=RND(2)*19:NEXT:NEXT:FORX=5TO7:
  V(X,4)=X+11:NEXT:X=0:Y=0
11 REM Scroll Routines
12 FORX1=XTOX+7:FORY1=YTOY+5:Z=64+(X1-X)+8*(Y1-Y):SPRITE$(Z)=A
  (V(X1,Y1)):NEXT:NEXT
14 DR=STICK(0):ONDRGOSUB20,24,26,30,32,36,38,42:IFSTRIG(0)GOTO
  44
16 IF INKEY$=CHR$(27)THENX=0:Y=0:GOTO12
18 IFDRGOTO12ELSEGOTO14
20 Y=Y-1:IFY<0THENY=0
22 RETURN
24 GOSUB20:GOSUB26:RETURN
26 X=X+1:IFX>XL-8THENX=XL-8
28 RETURN
30 GOSUB26:GOSUB32:RETURN
32 Y=Y+1:IFY>YL-6THENYL=YL-6
34 RETURN
36 GOSUB32:GOSUB38:RETURN
38 X=X-1:IFX<0THENX=0
40 RETURN
42 GOSUB38:GOSUB20:RETURN
44 REM Last Line

```

LISTING 3

By L A Dunning

```

1 REM Listing 3
2 REM Demonstrates manipulation of sprite tables
3 REM INITIALISATION
4 CLEAR2000:SCREEN1,0:DIMSS$(4):RESTORE:SA=&H1B00:SP=&H3800
5 FORA=0TO4:READA$:FORX=1TOLEN(A$)STEP2:S$(A)=S$(A)+CHR$(V
  AL("&H"+MID$(A$,X,2))):NEXT:NEXT
6 FORA=0TO4:SPRITE$(A)=S$(A):NEXT
7 PUTSPRITE0,(-4,8),15,2:PUTSPRITE1,(60,-4),1,1:FORA=2TO4:
  PUTSPRITEA,(32*A,32*A),1,A:NEXT
8 KEYON:ONKEYGOSUB100,200
9 LINE(0,0)-(255,55),6,BF:LOCATE0,0:PRINT"      Sprite   Y
  X      Pattern Tag":FORA=0TO4:SX=SA+A*4:PRINTUSING"
  ##      ###      ###      ###      ###";A,VPEEK(SX),VPEEK

```

```

      (SX+1),VPEEK(SX+2),VPEEK(SX+3):NEXT
10 PRINT"Press F1 or F2 to change size or mag."
11 IFNOT(STRIG(0))GOTO11ELSEGOTO40000
100 REM   Change size
101 GOSUB700:IFPVAND2THENPV=PVAND253ELSEPV=PVOR2
102 GOTO202
200 REM   change MAGNIFICATION
201 GOSUB700:IFPVAND1THENPV=PVAND254ELSEPV=PVOR1
202 GOSUB800:RETURN
700 PV=PEEK(&HFA07):RETURN
800 POKE&HFA07,PV:OUT129,PV:OUT129,129:RETURN
2000 DATA 708898A8C8887000,2060202020207000,708808304080F800,
      F808103008887000,55AA55AA55AA55AA
40000 REM   last line

```

These 3 listings are available from the BASIC PD library.

QUESTIONS & ANSWERS

In the February issue these questions were asked.

When a STAT *.* command is issued, there is a column of numbers headed Ext. What do they mean?

One of the options in the NSWEEP menu is Y - Set file status. I have tried quite a few things with this, and it doesn't seem to do anything. Does anyone know what this option is supposed to do?

Graeme Dick has supplied the answers to both questions. Firstly, you will recall Graemes article in the Nov 86 issue when he wrote about the disk utility DUV83, the 13th byte of a directory entry tells the extent of a file, and this is what this column of numbers refers to. Well what is an extent? you ask. Ok, some background information first. CP/M uses allocation groups, which on SV SS disks is 8 consecutive sectors. Each directory entry has a portion called an allocation group table which can have a maximum 16 allocation groups. If the file is larger than this, a second directory entry is required and the extent byte is set to 01 on the second entry. If the file is really big and still requires more, a third directory entry can be used and the extent byte is set to 02 on the third entry. And so on until the file is accomodated. So to summarize now, the Ext column tells us how many directory entries were required for a file.

The second question is a lot more straight forward. This option is used on tagged files, and that is why it would not work previously (because there were no files tagged). Now, having typed Y it comes up with Which flags (1-4,R,S,A)? These briefly are:

1-4 are used in machine code programs so probably will not be used by the majority. A file with any of these flags set is shown by having the first position of the file name in reverse video (RV) for flag 1, the 2nd, 3rd & 4th positions of the file name in RV for flags 2, 3 & 4 respectively.

R sets the tagged file(s) to R/O status. It does the same as STAT filespec \$R/O, but is a lot easier. A file with this flag set is shown by having the first position of the extension in RV.

S sets the file(s) tagged to System attribute. With this status, the file name(s) will not show up in the disk directory. STAT filespec \$SYS achieves the same result. A file with this flag set is shown by having the second position of the extension in RV.

A on its own sets the tagged file(s) to R/W with ARC. This is NOT the same as STAT filespec \$R/W as this does not have ARC. A file with this flag set is shown by having the third position of the extension in RV.

The above flags can be combined. So answering the Which flags? prompt with say LRSA will set the tagged file(s) to 1 R/O SYS ARC. I hope this gives a better understanding of this option. If it has raised other questions write in with them. One obvious question is "My NSWEEP doesn't have reverse video. How do I get it." will be answered next issue. For this you will need a copy of DU-V83. I got hold of a copy after reading Graeme Dick's article on it, and I don't know how I ever did without it.

Now, this issue's questions are about dBase from a local member and they are:

1 I want to change disks in drive B: and so I issue a reset b command after I have changed disks. This is supposed to reset the DOS bit map allowing uninterrupted use of dBase program. Unfortunately all I get is Bdos Err On b: R/O and one useless sector everytime I try and write to the new disk that I have put in the b drive. Can any one help with this one?

2 An application of the quit command is QUIT TO <com file list> but when I use the form QUIT TO WS I get this:

```
. QUIT TO WS
*** End run  dBASE II  ***
```

```
*** Syntax error ***
```

```
?
```

```
QUIT TO WS
```

```
Correct and retry? (y/n)N
```

```
I have tried all manner of things and am now at my wit's end. Can anyone Help?
```

If you know the answer to either of these questions, pickup a pen (NOW!) and write to: The Editors (Q & A)

PO Box 26 050
Newlands, Wellington
New Zealand

We are making this column a regular feature every issue, so lets have some more questions. This is one of the functions of the group (ie sharing information with each other), so come on every one.

PROGRAM From Brian Speirs.

3 DIMENSIONAL TIC-TAC-TOE

This program requires an 80 column card to display the board. As written here, the program also requires a disk drive for the data file.

The object of the game is to get 4 X's in a row through a cube with 4 squares to a side before the computer gets 4 O's in a row. The computer's strategy is very good but it can be beaten.

Under TURBO PASCAL, the computer takes about half a second to make its move. If converted to MBASIC, the program takes about 22 seconds to make its move. SV-BASIC would probably take slightly longer.

For a slightly more "random" strategy from the computer, the following section of code can be inserted at the end of Procedure Potential_Moves. ie just before the procedure 'end;'.

```
for Square := 1 to 64 do begin
  SqrValue := SqrVal[Square];
  if (SqrValue >= 4096) and (SqrValue <= 8192) then
    SqrVal[Square] := SqrValue - 4096;
end;
```

GOOD LUCK !

DATA FOR 3-DIMENSIONAL TIC-TAC-TOE

1	22	43	64	4	23	42	61	13	26	39	52	16	27	38	49
1	6	11	16	17	22	27	32	33	38	43	48	49	54	59	64
4	7	10	13	20	23	26	29	36	39	42	45	52	55	58	61
1	18	35	52	5	22	39	56	9	26	43	60	13	30	47	64
4	19	34	49	8	23	38	53	12	27	42	57	16	31	46	61
1	21	41	61	2	22	42	62	3	23	43	63	4	24	44	64
13	25	37	49	14	26	38	50	15	27	39	51	16	28	40	52
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64
1	5	9	13	2	6	10	14	3	7	11	15	4	8	12	16
17	21	25	29	18	22	26	30	19	23	27	31	20	24	28	32
33	37	41	45	34	38	42	46	35	39	43	47	36	40	44	48
49	53	57	61	50	54	58	62	51	55	59	63	52	56	60	64
1	17	33	49	2	18	34	50	3	19	35	51	4	20	36	52
5	21	37	53	6	22	38	54	7	23	39	55	8	24	40	56
9	25	41	57	10	26	42	58	11	27	43	59	12	28	44	60
13	29	45	61	14	30	46	62	15	31	47	63	16	32	48	64

304 records in file.

```

Program D3;
(* ----- *)
(* *)
(* ***** 3 DIMENSIONAL TIC TAC TOE PROGRAM ***** *)
(* *)
(* Converted from ADDS MENTOR DATA/BASIC by Brian Speirs, December 1986. *)
(* *)
(* ----- *)

```

```

type Lin = 1..76; (* Number of lines thru a 4x4x4 cube *)
Colv = 1..4; (* Number of squares in each line *)
Squarev = 1..64; (* Number of squares on 3-D board *)

var Board : array[Squarev] of Integer; (* 4x4x4 Board *)
RowVal : array[Lin] of Integer; (* Sum of counter values *)
SqrVal : array[Squarev] of Real; (* Move value of square *)
RowSqr : array[Lin,Colv] of Squarev; (* Squares in each line *)
List : array[Squarev] of Integer; (* List of max value moves *)

```

```

Value, Count, X, Y, Item, Position, Len : Integer;
RowValue, Sqr_Count, Err_Code, Data : Integer;
Temp, SqrValue, Max_Value : Real;
Line : Lin;
Level, Row, Column, Col : Colv;
Square, SqrCount : Squarev;
Err_Flag, Win, Quit, Turn, Valid : Boolean;
Dta_file : file of Integer;
Header : String[79];
Row1, Row2 : String[18];
Move : String[7];
Mark, Start, Character : Char;

```

```

procedure HouseKeeping;
begin

```

```

    Randomize;

```

```

    Err_Flag := false;
    Win := false;
    Quit := false;

```

```

    Header := '***** THREE DIMENSIONAL TIC-TAC-TOE ';
    Header := Header + '*****';
    Row1 := '---!---!---!';
    Row2 := '---!---!---!';

```

```

    assign(Dta_file, 'D3.DTA');
    reset(Dta_file);
    for Line := 1 to 76 do
        for Col := 1 to 4 do begin
            read(Dta_file, Data);
            RowSqr[Line, Col] := Data;

```

```

        end;
    close(Dta_file);

```

```

end;

```

```

Procedure Display_Board;
begin

```

```

    ClrScr;
    writeln(Header);
    writeln;
    write(Row2, Row2, Row2, Row2); writeln;
    write(Row1, Row1, Row1, Row1); writeln;
    write(Row2, Row2, Row2, Row2); writeln;
    write(Row1, Row1, Row1, Row1); writeln;
    write(Row2, Row2, Row2, Row2); writeln;
    write(Row1, Row1, Row1, Row1); writeln;
    write(Row2, Row2, Row2, Row2); writeln;

```

```

GotoXY(01,11); write('Your last move (X)');
GotoXY(40,11); write('Computer's last move (O)');
GotoXY(01,13);
writeln('  L R C                                L R C');
writeln(' E O O                                E O O');
writeln(' V W L                                V W L');
end;

Procedure Print_Move;
begin
  case Turn of
    true : begin
      Mark := 'X';
      Value := 1;
      end;

    false : begin
      Mark := 'O';
      Value := 10;
      end;

  end;

  Board[Square] := Value;
  Square := Square - 1;
  Level := Square div 16;
  Row := (Square - 16 * Level) div 4;
  Column := Square - 16 * Level - 4 * Row;
  X := 2 + 4 * Column + 18 * Level;
  Y := 3 + 2 * Row;
  GotoXY(X,Y); write(Mark);

  Turn := not Turn;
  Level := Level + 1;
  Row := Row + 1;
  Column := Column + 1;
end;

Procedure Instructions;
begin
  GotoXY(01,19);
  writeln('3D is played in a cube that is displayed on four planes. ');
  writeln('You enter moves by specifying - Level, Row, & Column. ');
  writeln('The above board shows a sample move. ');

  GotoXY(03,17); write('2 3 4');
  Turn := true;
  Square := 28;
  Print_Move;
end;

Procedure Choice;
begin
  Win := false;
  for Square := 1 to 64 do
    Board[Square] := 0;

  repeat
    Valid := true;
    GotoXY(01,23);
    write('Specify who starts - Man (M) Computer (C) End (E) ');
    readln(Start);
    Start := upcase(Start);

    case Start of
      'E' : begin
        Quit := true;
        Win := true;
        end;
    end;
  until Valid;

  (* Terminates Loop *)
end;

```

```

        'C' : begin
            Display_Board;
            Turn := false;
            Square := Random(63);
            Square := Square + 1;
            Print Move;
            GotoXY(41,17);
            write(Level, ' ', Row, ' ', Column);
            end;

        'M' : begin
            Display_Board;
            Turn := true;
            end;

    else
        Valid := false;
    end;
until valid;
GotoXY(01,19); write(#27 + 'J');  (* Clear to End of Screen *)
end;

Procedure Extract_Moves;
begin
    Err_Flag := false;
    Err_Code := 0;
    Len := length(Move);
    Level := 0;
    Row := 0;
    Column := 0;
    Position := 0;
    Count := 1;
    Item := 1;
    Valid := false;

    repeat
        Character := copy(Move, Count, 1);
        Val(Character, Position, Err_Code);

        if Err_Code = 0 then begin
            if (Position > 0) and (Position < 5) then
                case Item of
                    1 : Level := Position;
                    2 : Row := Position;
                    3 : Column := Position;
                end;
            Item := Item + 1;
            if Item = 4 then
                Valid := true;
            end;
            Count := Count + 1;
            if Count > Len then
                if not Valid then begin
                    Valid := true;
                    Err_Flag := true;
                end;
            until Valid;
        end;

    Procedure Get_Move;
    begin
        repeat
            GotoXY(01,17); write(' ');
            GotoXY(03,17); read(Move);
        until Valid;
    end;
end;

```

```

    if length(Move) < 3 then
        Err_Flag := true
    else
        Extract_Moves;

    if (Level = 0) or (Row = 0) or (Column = 0) then
        Err_Flag := true;

    if Err_Flag then begin
        GotoXY(01,21); write('***** Illegal Move *****');
        end
    else begin
        Square := (16 * Level) + (4 * Row) + Column - 20;
        if Board[Square] > 0 then begin
            GotoXY(01,21); write('***** Illegal Move *****');
            Err_Flag := true;
        end;
    end;
until not Err_Flag;

GotoXY(01,21); write(' ');
Print Move;
GotoXY(03,17); write(' ');
GotoXY(03,17); write(Level, ' ', Row, ' ', Column);
end;

Procedure Calc_Move_Value;
begin
    for Line := 1 to 76 do begin
        RowVal[Line] := 0;
        for Col := 1 to 4 do begin
            Square := RowSqr[Line,Col];
            RowVal[Line] := RowVal[Line] + Board[Square];
        end;
    end;
end;

Procedure Potential_Moves;
begin
    for Square := 1 to 64 do
        SqrVal[Square] := 0;

    for Line := 1 to 76 do begin
        RowValue := RowVal[Line];
        SqrValue := 0;

        if RowValue = 0 then SqrValue := 1;
        if RowValue = 1 then SqrValue := 8;
        if RowValue = 10 then SqrValue := 64;
        if RowValue = 2 then SqrValue := 512;
        if RowValue = 20 then SqrValue := 4096;
        if RowValue = 3 then SqrValue := 32768;
        if RowValue = 30 then SqrValue := 262144;
        if RowValue = 4 then begin
            Win := true;
            Line := 76;
        end;

        if not Win then begin
            for Col := 1 to 4 do begin
                Square := RowSqr[Line,Col];
                if Board[Square] = 0 then
                    SqrVal[Square] := SqrVal[Square] + SqrValue;
            end;
        end;
    end;
end;
end;

```

```
Procedure Select_Move;
```

```
begin
```

```
  if Win then Exit;
```

```
  Max Value := 0;
```

```
  Sqr_Count := 0;
```

```
  for Square := 1 to 64 do begin
```

```
    SqrValue := SqrVal[Square];
```

```
    if SqrValue > Max Value then begin
```

```
      Max Value := SqrValue;
```

```
      Sqr_Count := 1;
```

```
      List[Sqr_Count] := Square;
```

```
    end else
```

```
      if SqrValue = Max Value then begin
```

```
        Sqr_Count := Sqr_Count + 1;
```

```
        List[Sqr_Count] := Square;
```

```
      end;
```

```
  end;
```

```
  Item := Random(Sqr_Count);
```

```
  Item := Item + 1;
```

```
  Square := List[Item];
```

```
  Print Move;
```

```
  GotoXY(41,17); write(' ');
```

```
  GotoXY(41,17); write(Level, ' ', Row, ' ', Column);
```

```
end;
```

```
Procedure Game_Over;
```

```
begin
```

```
  if Win then begin
```

```
    GotoXY(01,19); write('YOU WIN !!!!!!!');
```

```
  end;
```

```
  if Max Value >= 262144. then begin
```

```
    GotoXY(01,19); write('COMPUTER WINS !!!!!!!');
```

```
    Win := true;
```

```
  end;
```

```
  if Win then begin
```

```
    GotoXY(01,21);
```

```
    write('Do you wish to play again ?? ');
```

```
  end;
```

```
end;
```

```
BEGIN
```

```
  HouseKeeping;
```

```
  Display Board;
```

```
  Instructions;
```

```
  while not Quit do begin
```

```
    Choice;
```

```
    while not Win do begin
```

```
      Get Move;
```

```
      Calc Move Value;
```

```
      Potential Moves;
```

```
      Select Move;
```

```
      Game_Over;
```

```
    end;
```

```
  end;
```

```
  Clrscr;
```

```
END.
```

MAIL ORDER FORM

MSX Computers & Software
11 Bradpole Rd., Elizabeth West, S.A. 5113

Please Supply:-_____

Please inc.\$2:00 for orders under \$30:00____\$____:

TOTAL _____ \$ _____ : _____

I enclose Cheque/Money Order for \$____:____
Payable to MSX Computers & Software

OR

Charge my Bankcard/Mastercard/Visa with \$____:

M/card or Visa:

--	--	--	--

Bankcard:

--	--	--	--

Please print clearly.

Card Expiry Date_____

Cardholder's Signature_____

Name: _____

Address: _____

State: _____ Postcode: _____

Computer make, type & memory (eg. 32K, 64K) _____

We are always looking for good, original commercial software for distribution.

ADVENTURE

Journey to the Centre of the E	12.95	Castle Blackstar	29.95	The Wreck	29.95
The Wreck (CART)	39.95	Mandragore	59.95	Vera Cruz	39.95
Murders on the Atlantic	39.95	The Inheritance	39.95	Lords of Time	39.95
Emerald Isle	29.95	Red Moon	29.95	Secret Diary of Adrian Mole	39.95
The Price of Magik	39.95	Special Operations	34.95	Panzer Attack	34.95
The Archers	39.95	Classic Adventure	29.95	Zakkil Wood	29.95
Jewels of Darkness	64.95	Silicon Dreams	64.95		

RECREATIONAL

Print X Press	29.95	Print X Press (DISC)	29.95	Jump Jet	44.95
Flight Path 737	39.95	Musix	34.95	Musix (DISC)	39.95
Flight Deck	44.95	Flight Deck (DISC)	49.95	North Sea Helicopter	44.95
North Sea Helicopter (DISC)	49.95	The Chess Game	34.95	The Chess Game (DISC)	44.95
Confused?	39.95	Confused? (DISC)	49.95	Dawn Patrol	49.95
Dawn Patrol (DISC)	59.95	Contract Bridge	39.95	Boardello	14.95
Pro-Snooker	9.95	Steve Davis Snooker	39.95	Backgammon	19.95
MSX Artist	19.95	Footballer of the Year	34.95	Supermaze	29.95
Super Puzzle	29.95	Patience	29.95	Cubit	29.95
737 Flight Simulator	44.95	Star Seeker	9.95	Spitfire 48	39.95
Gunshoe	39.95	Harvey Smith's Show Jumper	29.95	The Boss	29.95
Boardgames	19.95	Odyssey K	49.95	PSG Musewriter (CART)	89.95
FM Musicwriter (CART)	99.95	Cluedo	44.95	Monopoly	44.95
Scrabble	44.95	3D-Golf	14.95	Chess (CART)	29.95
Backgammon (CART)	24.95	Bridge	49.95		

ARCADE

Chuckie Egg	19.95	Slap Shot	14.95	Oh No!	14.95
Hopper	14.95	Scentipede	14.95	Boon	14.95
Jet Bomber	29.95	Jet Bomber (DISC)	44.95	Oils Well	34.95
Oils Well (DISC)	44.95	Time Curb	39.95	Time Curb (DISC)	44.95
The Heist	34.95	Space Rescue	14.95	Mac Attack	14.95
Panel Panic	14.95	Moonrider	14.95	Space Busters	14.95
Snake It	14.95	Ice	14.95	Mazes Unlimited	14.95
Snack Wacker	14.95	Robot Wars	14.95	Eggy	39.95
Meaning of Life	34.95	Fuzzball	34.95	Apeman Strikes Again	34.95
Spy Story	39.95	Science Fiction	39.95	Star Wars	34.95
City Connection	29.95	Formation 2	29.95	D-Day	29.95
Top Roccer	29.95	Ninja 1	29.95	Ninja 2	29.95
Snake Runner	29.95	Zorny	29.95	Exterior	29.95
Chopper 1	34.95	Police Academy	34.95	Compilation 5	39.95
Compilation One (DISC)	44.95	Oh Shit!	14.95	Panique	14.95
Compilation 4	39.95	Supersellers I	39.95	MSX Extra	39.95
Beamrider	39.95	River Raid	39.95	H.E.R.O	39.95
Pitfall II	39.95	Ghostbusters	39.95	Decathlon	39.95
Master of the Lambs	39.95	Hustler	14.95	Starquake	34.95
Wizards Lair	34.95	Turmoil	12.95	Ole	12.95
Chicken Chase	12.95	Zoot	12.95	Time Trax	12.95
10 Computer Hits	44.95	5 Star Games	44.95	Vampire	9.95
Octagon Squad	9.95	Ice King	29.95	A View to a Kill	29.95
International Karate	29.95	Shark Hunter	19.95	Buzzoff	9.95
Norseman	19.95	Le Mans	19.95	Barnstormer	19.95
Chack n Pop	19.95	Choro Q	19.95	XYZolog	19.95
Sweet Acorn	19.95	Front Line	19.95	The Way of the Tiger	39.95
Jack the Nipper	39.95	Trail Blazer	39.95	Avenger	39.95
MSX Classics	44.95	Desolator	34.95	Attack of the Killer Tomatoes	29.95
Mr Ching (CART)	49.95	Step Up (CART)	49.95	Rollerball (CART)	49.95
Super Billiards (CART)	49.95	Hole in One (CART)	49.95	Super Snake (CART)	49.95

ARCADE

Eggerland (CART)	39.95	Space Trouble (CART)	49.95	Space Maze Attack (CART)	49.95
Fruit Search (CART)	49.95	Picture Puzzle (CART)	49.95	Heavy Boxing (CART)	49.95
Hole in One Professional (CART)	49.95	Dunkshot (CART)	49.95	Butu Maru (CART)	49.95
Green Beret (CART)	49.95	Super Cobra (CART)	49.95	Athletic Land (CART)	49.95
Circus Charlie (CART)	39.95	Antartic Adventure (CART)	49.95	Conic Bakery (CART)	49.95
Monkey Academy (CART)	39.95	Time Pilot (CART)	49.95	Hyper Sports I (CART)	49.95
Hyper Sports II (CART)	49.95	Track & Field I (CART)	39.95	Track & Field II (CART)	39.95
Konami's Tennis (CART)	49.95	Tie Ar Kung Fu (CART)	49.95	Sky Jaguar (CART)	49.95
Mopiranger (CART)	39.95	Konami's Golf (CART)	49.95	Road Fighter (CART)	49.95
Ping Pong (CART)	49.95	Boxing (CART)	49.95	Konami's Soccer (CART)	49.95
Tie Ar Kung Fu II (CART)	49.95	Hyper Rally (CART)	49.95	Hyper Sports III (CART)	49.95
Knightmare (CART)	49.95	Konami's Billiards (CART)	49.95	Nemesis (CART)	49.95
The Goonies (CART)	49.95	Jailbreak (CART)	49.95	Kings Valley (CART)	49.95
Skramble	9.95	Gridtrap	9.95	Alpha Blaster	9.95
Salvage	9.95	Congo	9.95	Juapin' Jack	9.95
Invaders	9.95	Can Of Worms	9.95	Space Walk	9.95
Chiller	9.95	Formula 1 Simulator	9.95	Speed King	9.95
Storm	9.95	Video Poker	9.95	Soul of a Robot	9.95
Crazy Golf	29.95	Punchie	29.95	Humphrey	29.95
Mayhem	29.95	Dynomite Dan	34.95	Hunchback	34.95
Foot Volley	9.95	Vestron	9.95	Manic Miner	29.95
Jet Set Willy II	29.95	Les Flics	34.95	Macadam Bumper	39.95
Boulderdash	29.95	Brian Jacks - Super Challenge	29.95	Eddie Kidd - Jump Challenge	4.95
Zoids	34.95	Lazy Jones	34.95	Winter Olympics	14.95
Dan Busters	39.95	Cyberun	39.95	Winter Games	39.95
Unbelievable Ultimate	44.95	Sorcery	29.95	Kick It	14.95
Battleship Clapton II	14.95	Pyramid Warp	14.95	Pinball	14.95
Polar Star	14.95	Crazy Train (CART)	29.95	Mouser (CART)	29.95
Alibaba and 40 Thieves (CART)	29.95	Dorodon (CART)	29.95	Senjo (CART)	29.95
Choplifter	29.95	Loadrunner (CART)	34.95	Alpha Squadron (CART)	29.95
Star Blazer (CART)	29.95	Raid On Bungeling Bay (CART)	34.95	Cosmo Explorer (CART)	34.95
Payload (CART)	34.95	Donkey Kong	39.95	Who Dares Wins	29.95

EDUCATIONAL

French is Fun	34.95	MUE (CART)	79.95	Introduction to Nuabers	39.95
Calculation I	39.95	Calculation 2	39.95	Reflexes	39.95
First Steps With the Mr Men	39.95	Here and There with the Mr Men	34.95	Simple Addition I	34.95
Introducing Circle I	34.95	Introducing Circle II	34.95	Trigonometry Basic Concepts	34.95
Introducing Percentages I	34.95	Subtraction I	34.95	Introducing Fractions II	34.95
Introduction to Percentages II	34.95	Introducing Sequences	34.95	Introduction to shapes	34.95
Introducing the Rectangle	34.95	Introducing the Square	34.95	Introducing the Triangle	34.95
Subtraction II	34.95	Tens & Units	34.95	Introducing Symmetry	34.95
Introducing the Angle	34.95	Revise Physics	34.95	Teach Yourself Electricity	39.95
O Level Maths Examiner	39.95	O Level Physics Examiner	39.95	O Level Chemistry Examiner	39.95
Maths I	39.95				

BUSINESS

Cash Accounts (CART)	139.95	MSX Calc (CART)	169.95	The Secretary (CART)	299.95
MSX Text (WP & CARDS)	169.95	MT Base	139.95	Spreadsheet	99.95
Tasword	69.95				

UTILITY

Wordstore Plus (DISC)	49.95	Aackodraw & Paint	39.95	Aackodraw & Paint (DISC)	49.95
Debugger (DISC)	64.95	MSX Logo (CART)	199.95	Operation Alignment	29.95
EDDY II (CART)	79.95	Hi-Soft Devpac	74.95	Devpac 80 (DISC)	139.95
Pascal 80 (DISC)	139.95	C+ (DISC)	139.95	Champ	64.95

ARCADE AND ADVENTURE

Minder	29.95	Finders Keepers	9.95	Molecule Man	9.95
Knightyme	14.95	Battle for Midway	39.95	Batman	39.95

BOOKS

Getting Started on Your MSX	29.95	MSX Adventure Programming	34.95	MSX Applications	34.95
Getting More From MSX	34.95	Introducing MSX BASIC	29.95	MSX	34.95
Starting With The MSX	29.95	Starting Machine Code	34.95	Programming MSX Basic	34.95
Ideas for MSX	29.95	Practical MSX M/C Programming	19.95	Games for your MSX	19.95
Useful utilities for your MSX	19.95	MSX The Standard	69.95		

HARDWARE

Zeta Data Recorder	39.95	Toshiba Dot Matrix Printer	399.00	Quickshot I (MSX) Joystick	22.95
Quickshot II (MSX) Joystick	32.95	Quickshot Joystick (MSX)	14.95	Viatel Modem with Software	399.00
Blank 3 1/2 inch Disk	7.50	Blank 3 1/2 inch disks (10)	69.95	6V Power Supply (Suit Zeta)	16.95

Public Domain CP/M

CLASSIFIEDS

WANTED TO BUY

PASCAL COMPILER Suitable for use on 328 Mk II with two disk drives. If you have one you would like to sell please contact:- David Burrowes, 25 Montrose Drive, NELSON.

WANTED TO SELL

V-328 MKI AND 904 DATA CASSETTE PLUS 601 6 Slot Expander (old style) with Centronics interface Disk Controller A\$1,125 plus postage. If interested please contact:- Jim Collins, 1 Conrad Avenue, Georgetown, TASMANIA 7253 PH: Georgetown (03) 822 919 Hm

SV-601 Expander (old style) Plus SV-902 Disk Drive SV-802 Centronics interface Ideal for anyone wishing to upgrade to a disk based system. Software included. Phone Brian Speirs 722-178 (w) or 764-765 (h) for information.

SV-328 MK II WITH 904 Data Cassette, 902 Drive, 601 Expander, 2 Joy Sticks, Zenith Green Screen and software (no centronics and no 80 column card) \$1,200 o.n.o. Contact Grant Stone Ph 326-280

WANTED TO SELL

SV-328 COMPUTER and 904 date cassette WITH Expander with 2 88 Disk Drives and 80 Column Card, 1 Star-gemini 10X Printer, Complete set Micropro CP/M Programmes including: Wordstar, Calcstar, Mailmerge etc with full manuals. Would prefer to sell as one lot but would consider selling items separately depending on offers. Phone Peter Bradley Hastings (070) 797-240 evenings
